

ACADEMIC TITANS HACKATHON 2026 · AT-01-S4

CodeFix

AI-Based Coding Tutor for Engineering Students

Project Documentation

Stanley College of Engineering & Technology for Women, Hyderabad

HACKATHON
Academic Titans**LANGUAGES**
Python · C · C++**SDG ALIGNMENT**
SDG 4 · Quality Education

Team SheCoders

Aastha Vyas · Amreen Fatima · Nabila Tahoore

Stanley College of Engineering & Technology for Women, Hyderabad

"Stack Overflow tells you what to paste. CodeFix tells you why it broke."

Table of Contents

1.	Problem Statement	3
2.	Project Overview	3
3.	Software Development Life Cycle (SDLC)	4
4.	System Architecture	5
5.	Technical Approach & Implementation	6
6.	Key Features	8
7.	Error Handling & Resilience	9
8.	Optimization Milestones	10
9.	UN SDG Alignment	11
10.	Team & Conclusion	11

1. Problem Statement

Engineering students in their 1st and 2nd year frequently encounter coding errors and conceptual gaps while learning Python, C, and C++. The standard debugging workflow — reading a cryptic error, searching Stack Overflow, copying a fix — teaches them what to paste but never why it was wrong.

This creates a critical gap: students who can fix bugs but cannot prevent them. Personalized tutoring is either unavailable or unaffordable for most students, especially in developing regions.

The Problem	CodeFix Solution
- Tools only fix code, not understanding - No personalized tutor available 24/7 - Same mistakes repeated by students - Financial barrier to paid mentorship	- Diagnoses exact error + explains WHY - Available 24/7, free, browser-based - Remember This tip prevents repetition - Completely free — no subscriptions

2. Project Overview

CodeFix is an AI-powered coding tutor built specifically for 1st and 2nd year engineering students learning Python, C, and C++. Unlike generic AI tools that simply fix code, CodeFix teaches students the root cause of every error, making them better programmers over time.

2.1 Hackathon Details

Hackathon	Academic Titans
Problem Statement	AT-01-S4 — AI-Based Student Doubt Solving Chatbot
Team	SheCoders
Institution	Stanley College of Engineering & Technology for Women, Hyderabad
Final Version	v5.0 — app.py

2.2 Tech Stack

Component	Technology	Type	Role
Frontend	Streamlit	Python Framework	User Interface & 3-Tab Layout
AI Engine	Groq API (llama-3.3-70b)	LLM API	Code Analysis & Explanations
Backend	Python 3.x	Language	Core Logic & Code Execution

Code Runner	subprocess module	Standard Library	Run Python/C/C++ in isolation
Prompting	CRISP Framework	Methodology	Structured AI prompt design

3. Software Development Life Cycle (SDLC)

CodeFix followed an Agile-inspired SDLC over 4 weeks, with each week representing a sprint with defined deliverables. The development evolved from a basic prototype to a polished, production-quality application.

3.1 Phase 1 — Requirements (Week 1)

The team identified the core problem, defined user personas, and documented functional and non-functional requirements.

Functional Requirements

- FR-01: Accept and execute student code automatically (Python, C, C++) without manual error pasting
- FR-02: Capture real stdout and stderr from code execution via subprocess
- FR-03: Send code + captured output to Groq AI with CRISP-structured prompt
- FR-04: Return structured 5-section response: Error Diagnosis, Why This Happened, Fixed Code, Code Improvement, Remember This
- FR-05: Support Concept Doubt mode for theoretical explanations with analogy and example
- FR-06: Persist doubt history within session; viewable in History tab
- FR-07: Support Beginner and Intermediate levels with adaptive tone

Non-Functional Requirements

- NFR-01: Response time under 15 seconds for AI-powered answers
- NFR-02: Graceful error handling — no raw tracebacks shown to students
- NFR-03: Clean, distraction-free UI with dark theme optimized for coding
- NFR-04: API rate limit resilience — automatic retry with user notification

3.2 Phase 2 — Design (Week 1–2)

The team designed the system architecture, prompt structure using the CRISP framework, and the UI layout. Key design decisions included:

- 3-tab layout (Home / Solve / History) to separate concerns and reduce clutter
- CRISP prompt framework to ensure consistent, structured AI responses every time
- Session state management using `st.session_state` for history persistence without a database
- Subprocess isolation for safe code execution in a child process

3.3 Phase 3 — Implementation (Week 2–3)

Development proceeded through 5 major versions, each addressing specific judge feedback or discovered issues. See Section 8 (Optimization Milestones) for the full refactoring log.

3.4 Phase 4 — Testing & Resilience (Week 3)

Every realistic failure mode was identified and handled. See Section 7 (Error Handling & Resilience) for the complete test coverage table.

3.5 Phase 5 — Deployment & Documentation (Week 4)

Final polish, README completion, GitHub repository organization, and this documentation were produced in Week 4. The app is deployable via Streamlit Community Cloud or run locally with a single command.

4. System Architecture

CodeFix is structured as a 5-layer pipeline. Each layer has a single, well-defined responsibility.

Layer	Component	Technology	Role
01 · User	Student	Browser / Streamlit App	Pastes code or types concept doubt
02 · Frontend	Streamlit · 3 Tabs	Python + Custom CSS	Home, Solve, History tabs + session state
03a · Execution	Code Runner	subprocess + tempfile	Runs code in isolated child process, captures stdout/stderr
03b · Prompt	CRISP Engine	build_code_prompt / build_concept_prompt	Structures AI prompt with exact headings and context
04 · AI	Groq API	llama-3.3-70b-versatile	Generates structured 5-section response with retry logic
05 · Output	Response Cards	parse_sections + st.markdown	5 visual cards + auto-save to History tab

4.1 Data Flow

The request lifecycle for a Code Doubt submission follows this sequence:

1. Student pastes code into the editor panel and clicks Run & Debug
2. Streamlit validates that the code field is not empty (st.stop() guard)
3. run_code() writes code to a temp file and executes it via subprocess.run()
4. stdout and stderr are captured from the child process
5. build_code_prompt() constructs the CRISP-structured prompt with code + output
6. ask_gemini() (Groq API) sends the prompt with retry logic on rate limits
7. parse_sections() extracts the 5 bold-headed sections from the AI response
8. Response breakdown is stored in st.session_state.breakdown and rendered as cards
9. Entry is appended to st.session_state.history for the History tab

5. Technical Approach & Implementation

5.1 CRISP Prompt Framework

CodeFix uses the CRISP framework (Context, Role, Instructions, Specifics, Parameters) to structure every AI prompt. This ensures consistent, high-quality responses regardless of student input quality.

Element	Stands For	How CodeFix Uses It
C	Context	Student level (Beginner/Intermediate), language (Python/C/C++), code + real execution output
R	Role	"You are CodeFix, an AI coding tutor for 1st/2nd year engineering students"
I	Instructions	Diagnose, explain root cause, provide fix, suggest improvement, give memorable tip
S	Specifics	Use EXACTLY these bold headings: Error Diagnosis, Why This Happened, Fixed Code, Code Improvement, Remember This
P	Parameters	Adapt explanation depth and language complexity based on Beginner vs Intermediate level

5.2 Code Execution Engine

CodeFix runs student code automatically — the student never needs to manually copy and paste error messages. The execution engine handles Python, C, and C++:

- Python: Written to a temp .py file, executed with python3, temp file deleted after run
- C: Compiled with gcc to a .out binary, executed, both source and binary cleaned up
- C++: Compiled with g++ to a .out binary, executed, both source and binary cleaned up
- Timeout: 10-second limit via subprocess.run(timeout=10) prevents infinite loops
- Isolation: Each execution runs in a separate child process — crashes never affect the main app

5.3 Response Parsing

The `parse_sections()` function uses regex to reliably extract each bold-headed section from the AI response:

- Pattern: `r"\\*(.+)\\*\\s*\\n(.*)?(?=\\n\\*\\*|\\Z)"` — matches any **Heading** block
- Code fence stripping: Removes ````python ... ```` markers so `st.code()` renders clean syntax highlighting
- Fallback: `secs.get('Key', '—')` ensures missing sections never crash the UI

5.4 Session State & History

Streamlit's `st.session_state` is used to persist data across reruns without a database:

- `st.session_state.history`: List of all doubt entries (type, language, level, input, output, response, time)
- `st.session_state.breakdown`: Current response — kept separate so input and output can coexist on the same page
- Auto-save: Every successful submission appends to history automatically, no student action required

6. Key Features

Feature	Description	Student Benefit
Live Code Execution	Student pastes code; CodeFix runs it automatically and captures real stdout + stderr	No manual error copy-paste needed
Error Diagnosis	Pinpoints exact bug and line number from the real execution output	Instant, precise error location
Why This Happened	Explains root cause in student-appropriate language (Beginner vs Intermediate)	Deep understanding, not just a fix
Fixed Code	Complete, corrected version of the student's code with syntax highlighting	Ready-to-use solution
Code Improvement	One concrete suggestion for a better coding habit beyond just fixing the bug	Builds better coding skills
Remember This	One memorable sentence the student can internalize to never repeat the mistake	Long-term retention
Concept Doubt Mode	Explains theoretical concepts with an example, real-world analogy, and exam tip	Covers both code and theory
Adaptive Levels	Beginner gets plain language + analogies; Intermediate gets technical depth	Meets every student where they are
Doubt History	Every session's doubts auto-saved; filterable by Code / Concept in History tab	Review and study anytime
3-Language Support	Python, C, and C++ all supported with language-specific execution and syntax	Covers full 1st-2nd year curriculum

6.1 Comparison with Existing Approaches

Feature	Stack Overflow	Generic AI (ChatGPT)	CodeFix
Auto code execution	✗	✗	✓
Root cause explanation	Sometimes	Inconsistent	✓ Always

Adaptive to student level	✗	✗	✓ Beginner/Intermediate
Session history	✗	✗ (new chat = lost)	✓ Auto-saved
Remember This tip	✗	✗	✓ Every response
Free, no account needed	✓	Limited free tier	✓ Completely free

7. Error Handling & Resilience

CodeFix was designed to never crash from student input or external failures. Every realistic failure scenario was identified and handled gracefully.

Failure Scenario	How CodeFix Handles It
Empty code box submitted	st.warning() shown, execution blocked with st.stop()
Empty concept doubt submitted	st.warning() shown, API call blocked with st.stop()
Code runs clean (no errors)	stderr shows '(no errors)', AI still provides code improvement tips
Python not installed on machine	subprocess catches FileNotFoundError, returns friendly message
gcc / g++ compiler missing	Returns 'Install MinGW / build-essential' guidance message
Code runs > 10 seconds	subprocess timeout=10 kills process, returns TimeoutError message
Groq API rate limited (429)	Auto-retry up to 3x with 20s/40s/60s exponential wait + toast notification
Groq API timeout (60s)	requests timeout=60 caught by try/except, retried up to 3 times
AI response missing a section	secs.get('Key', '—') fallback — page still renders without crashing
Markdown code fences in response	Regex strips ```lang and ``` before st.code() display

7.1 Key Resilience Patterns

- **Validation with st.stop() is called immediately if required inputs are empty, preventing blank API calls:** Input Guards
- **Code is written to a temp file, executed in a child process, file deleted after — main app never crashes from student errors:** Subprocess Isolation
- **Groq rate limits (429) trigger exponential wait-and-retry with user-visible toast notification:** API Retry Loop
- **Every response section uses secs.get('Key', '—') so a missing AI section never breaks the UI:** Graceful Fallbacks
- **Regex removes markdown code fences from AI output before rendering to st.code():** Fence Stripping

8. Optimization Milestones

The codebase evolved through 5 major versions over 4 weeks. Each refactor was driven by judge feedback, discovered bugs, or UI improvements.

8.1 Version History

Version	Change Made	Why
v1.0	Basic code + error paste, Gemini API	Initial prototype — proof of concept
v2.0	Added CRISP prompt framework	Inconsistent AI responses fixed with structured prompting
v3.0	Added Beginner/Intermediate level selector	Students needed adaptive explanations, not one-size-fits-all
v4.0	Live code execution via subprocess	Judges: remove manual error paste — run code automatically
v4.1	Removed manual error box	Redundant after auto-execution was added in v4.0
v4.2	Sidebar removed, full-width layout	Cleaner UI, no duplicate history panel
v4.3	Switched Gemini → Groq API	Free tier, faster inference, no quota resets
v4.4	Response cards stacked vertically	Less cramped layout, easier to read on smaller screens
v4.5	Markdown fence stripping in parser	Fixed ```python showing as raw text in code blocks
v5.0	3-tab layout: Home / Solve / History	Judges: students need to track doubts without re-running

8.2 Architecture Shift Summary

Before (v1–v2): Single-page layout, student manually pastes code AND error message, Gemini API, no history tracking.

After (v5.0): 3-tab app (Home/Solve/History), code executes automatically, errors captured by subprocess, Groq API with exponential retry, structured 5-card response breakdown, full session history with filter.

- **Separating the input view from the breakdown view using `st.session_state.breakdown` keeps the UI clean and prevents page clutter:** Key structural decision
- **CRISP framework enforces structured AI responses with exact bold headings, making `parse_sections()` reliable and UI rendering consistent:** Prompt improvement

9. UN SDG Alignment

SDG 4 — Quality Education

CodeFix directly supports United Nations Sustainable Development Goal 4: Quality Education — ensuring inclusive and equitable quality education and promoting lifelong learning opportunities for all.

Engineering students in developing countries often lack access to personal tutors or paid mentorship platforms. CodeFix bridges this gap by providing an AI-powered coding tutor that is completely free and accessible to anyone with an internet connection.

SDG Target	How CodeFix Addresses It
Target 4.1	Free, quality education for all students regardless of financial background — CodeFix is free to use, browser-based, no subscription required
Target 4.4	Increase youth with relevant technical and coding skills for employment — CodeFix builds real debugging and programming intuition, not just fix-and-forget habits
Target 4.7	Ensure all learners acquire knowledge needed for sustainable development — CodeFix uses the Remember This feature to build lasting conceptual understanding

10. Team & Conclusion

10.1 Team SheCoders

Name	Role	Contribution
Aastha Vyas	Lead Developer	Full-stack development, UI design, AI integration, architecture
Amreen Fatima	Documentation & Testing	Technical documentation, test case design, error scenario mapping
Nabila Tahoor	Research & Prompt Design	CRISP framework research, prompt engineering, SDG mapping

10.2 Setup Instructions

To run CodeFix locally:

10. Clone the repository: `git clone https://github.com/aasthavyas0023-web/codefix.git`
11. Install dependencies: `pip install streamlit requests`
12. Add your Groq API key in `app.py` (line ~387): `GROK_API_KEY = "gsk_your_key_here"`
13. Run the app: `streamlit run src/app.py`

14. Open in browser at: <http://localhost:8501>

For C/C++ support: install gcc/g++ via MinGW (Windows) or `sudo apt install build-essential` (Linux/Mac).

10.3 Conclusion

CodeFix addresses a real, widespread problem in engineering education — the gap between "I fixed it" and "I understand why it broke." By combining live code execution, structured AI explanations, adaptive levels, and session history, CodeFix acts as a 24/7 personal coding tutor for every student who cannot afford one.

Built in 4 weeks by Team SheCoders at Stanley College of Engineering and Technology for Women, Hyderabad, CodeFix demonstrates that meaningful AI-powered education tools can be built quickly, deployed for free, and designed to genuinely improve student outcomes.

"Stack Overflow tells you what to paste. CodeFix tells you why it broke."

CodeFix · Built with care by SheCoders · Academic Titans 2026